

Appendix D — Data Files and Reference Outputs

Not every artifact named in this appendix is what its filename suggests today. ``interactions.jsonl``, ``failed_variants.json``, and ``user_thumbsdowns.json`` are real names from the project's early flat-file design, retired on 2026-06-27 when ADR-046 moved all three into MongoDB collections — the ``jsonl``/``json`` names now survive only as documentation shorthand and in a handful of stale code comments. ``learned_qa`` is a live ChromaDB collection, never a file. ``rag_graph.png`` and the four log artifacts are literal files on disk. Table D.1 is the map from `Book_Index.md`'s nine names to what each one actually is in the code running today; the sections after it give each one's real schema.

Named artifact	Real, current form	Written by
<code>interactions.jsonl</code>	MongoDB <code>`feedback_interactions`</code>	<code>`feedback_store.py::log()`</code>
<code>failed_variants.json</code>	MongoDB <code>`failed_variants`</code>	<code>`feedback_store.py::save_failed_variants()`</code>
<code>user_thumbsdowns.json</code>	MongoDB <code>`user_thumbsdowns`</code>	<code>`feedback_store.py::_append_thumbsdown()`</code>
<code>learned_qa</code>	Live ChromaDB collection	<code>`self_learner.py`</code> , <code>`learned_qa_store.py`</code>
<code>rag_graph.png</code>	Real PNG, regenerated on topology change	<code>`app_workflow/main.py`</code>
<code>langchain_api_*.debug.log</code>	Real files under <code>`Runs/`</code>	<code>`app/`</code> 's <code>`logger_config.py`</code> + <code>`api.py`</code>
<code>langgraph_api_*.debug.log</code>	Real files under <code>`Runs/`</code>	<code>`app_workflow/services/logger_config.py`</code> + <code>`api.py`</code>
<code>old_log.txt</code> / <code>new_log.txt</code>	Real console-capture pair under <code>`Runs/`</code>	Manual capture, not code-generated

D.1 `interactions.jsonl` → MongoDB `feedback_interactions`

Every answered query — good, bad, or a no-context fallback — is logged as one document, ``insert_one()``-wrapped in a ``try/except DuplicateKeyError`` for idempotency, since LangGraph's own retry infrastructure can re-invoke the logging call for the same request:

```
record = {
    "ts": datetime.now(timezone.utc).isoformat(),
    "request_id": request_id or str(uuid.uuid4()),
    "query": query,
    "answer": answer,
    "quality": quality,
    "sources": sources,
    "document_chunks": self._stored_chunks(document_retrieved_chunks),
    "learned_qa_chunks": self._stored_chunks(learned_qa_retrieved_chunks),
    "variants": variants or [],
```


`quality` is a controlled vocabulary, not free text: `"OK"`, a grounding-judge verdict (`"PARTIALLY\_FABRICATED"`, `"OVERCLAIMED"`, `"OFF\_TOPIC"`, `"UNKNOWN"`), `"USER\_THUMBSDOWN"`, or `"INSUFFICIENT"` on the no-context path. `SelfLearner.run\_distillation()` queries exactly `{"quality": "OK"}`, sorted newest-first, as the pool of interactions eligible for distillation into `learned\_qa` (§D.4) — which is why `quality` is the one field this schema treats as load-bearing rather than descriptive.

Common pitfall — A comment that never got the memo

`app\_workflow/nodes/no\_context\_answer.py` still carries the comment `# Log the no-context interaction to interactions.jsonl as INSUFFICIENT.` — written before ADR-046's Mongo migration and never updated. The code beneath it is correct; the comment is not. A small, honest example of documentation surviving a refactor that the code it describes did not.

D.2 failed_variants.json → MongoDB failed_variants

One document per normalized question, holding every query rewording that retrieved zero useful chunks for it. The complete, real MongoDB command and its complete, real reply, captured from `Runs/langchain\_api\_20260710\_115823.debug.log` at the moment this exact question was asked a second time:

```
command: {"find": "failed_variants", "filter": {"normalized_query": "what
can u tell me about asd?"}, "limit": 1, "singleBatch": true, "$db":
"rag_db"}

reply: {"cursor": {"firstBatch": [{"_id": {"$oid":
"6a3fb8b5175314a656d7ce06"},
      "normalized_query": "what can u tell
me about asd?",
      "variants": ["What can u tell me about
ASD?",
      "ASD information that was
vague or imprecise"]}],
      "ns": "rag_db.failed_variants"}, "ok": 1.0}
```

```
def save_failed_variants(self, query: str, variants: list[str]) -> None:
    norm = self._normalize_query(query)
    self._failed_variants.update_one(
        {"normalized_query": norm},
        {"$addToSet": {"variants": {"$each": variants}}},
        upsert=True,
    )
```

`\$addToSet` is the whole point of this write: it appends new failing phrasings without a read-modify-write cycle and without ever duplicating a variant already recorded. This store closed a real gap recorded as BUG-043: the LangGraph pipeline originally read this collection at startup but had no node that ever wrote a newly-discovered failure back into it, so the blacklist could never grow.

The two variants above are not just stored — they are read back and injected into the very next attempt's system prompt, complete and verbatim, as this real excerpt from the same debug log's outbound LLM request body shows:

```
BLOCKED QUERY VARIANTS -- These exact phrasings retrieved 0 useful chunks
the last time this question was asked. Do NOT reuse them under any
circumstances. Generate different, semantically distinct reformulations:
- "What can u tell me about ASD?"
- "ASD information that was vague or imprecise"
- "ASD information"
- "Autism Spectrum Disorder facts"
- "ASD symptoms and causes"
```

Five phrasings appear here even though the stored document above holds only two — the other three were accumulated by `load_thumbdown_context()` from the matching `user_thumbdowns` record's own reformulation history (§D.3), confirming the two stores are merged into one combined blacklist at prompt-assembly time, not read independently.

D.3 `user_thumbdowns.json` → MongoDB `user_thumbdowns`

Written only when a user's typed feedback is at least 10 characters (`MIN_FEEDBACK_LEN`), and only inside a MongoDB session transaction shared with the matching interaction's `quality` update to `"USER_THUMBSDOWN"` (ADR-048) — so a crash mid-write can never leave a thumbdown recorded without its interaction updated, or vice versa. The complete, real reply MongoDB returned for this record (same debug log, same request) — note the `bad_answer` field ends mid-word: that truncation is the debug log's own logging cap on long string fields, present in the raw file itself, not shortened by this appendix:

```
{"_id": {"$oid": "6a3f9786f9cd661039734aa6"},
"ts": "2026-06-27T09:27:34.718895+00:00",
"request_id": "03c3566c-250c-46c4-815c-f9ec33ec1914",
"original_query": "What can u tell me about ASD?",
"normalized_query": "what can u tell me about asd?",
"bad_answer": "Autism Spectrum Disorder (ASD) is a complex
neurodevelopmental disorder characterized by
deficits in social communication and the presence of restricted
interests and repetitive behaviors
[Source: learned_qa]. The Diagnostic and Statistical Manual of Mental
Disorders-5th edition (DSM-5)
updated the diagnostic criteria for ASD in 2013, providing a more
accurate and comprehensive
understanding of the condition [Source: learned_qa]. This update
acknowledges the complexity and
diversity of ASD, emphasizing the need for more nuanced and
individualized approaches.

Individuals with ASD may experience a range of difficulties, including
challenges with verbal and
nonverbal communication, changes in routi..."}
```


This particular reply carries no `variants` field at all — a real example of a thumbsdown record predating full reformulation capture for this write path, not an omission by this appendix. `find\_thumbsdowns\_for\_query()` normalizes an incoming question and looks up prior thumbsdowns by exact string match; every hit is injected into the retrieval prompt as a complete “USER-FLAGGED PRIOR FAILURE” block. The real, complete injected block for this exact question — two prior failures, both surfaced together — reads:

```
USER-FLAGGED PRIOR FAILURE -- This same question (or a near-identical one)
was asked before and the user explicitly marked the answer as bad. Below is
what went wrong last time, including the exact query reformulations that
were tried and the chunks they retrieved.
```

1. AVOID re-using the same query reformulations -- they led to a bad answer.
2. AVOID grounding your answer in the same chunks unless you can clearly interpret them differently.
3. Try genuinely different angles, vocabulary, and sub-topics.

```
--- Prior failure #1 ---
```

```
Original query : What can u tell me about ASD?
```

```
User feedback  : The answer missed some information I believe.
```

```
Bad answer was : Autism Spectrum Disorder (ASD) is a complex
neurodevelopmental disorder characterized by deficits in social
communication and the presence of restricted interests and repetitive
behaviors [Source: learned_qa]. The Diagnostic and Statistical Manual of
Mental Disorders--5th edition (DSM-5) updated the ...
```

```
Reformulations that were tried (DO NOT REUSE):
```

```
- "ASD information"
```

```
retrieved learned_qa [learned_qa]: Q: How does the Diagnostic and
Statistical Manual of Mental Disorders--5th edition (DSM-5) define ASD? A:
Autism spectrum disorder (ASD) is a neurodevelopmental disorder
characterized by deficits in so...
```

```
retrieved learned_qa [learned_qa]: Q: What are the key
characteristics of ASD in the context of engineering and technology? A: ASD
is characterized by challenges in social communication, restricted
interests, repetitive behaviors, and ...
```

```
retrieved documents [C:\Users\LOQ\Desktop\Projects\RAG-
work\data\json\asd_information.json]: Autism Spectrum Disorder (ASD) is a
neurodevelopmental condition characterized by differences in communication,
social interaction, behavior patterns, and sensory processing.
```

```
retrieved documents [..\data\pdfs\Autism spectrum disorder
definition, epidemiology, causes, and clinical evaluation.pdf]: Autism
spectrum disorder (ASD) is a neurodevelopmental disorder characterized by
deficits in social communication and the presence of restricted interests
and repetitive behaviors (1). In 2013, the Di...
```

```
- "Autism Spectrum Disorder facts"
```

```
retrieved learned_qa [learned_qa]: Q: What are some common
characteristics and co-occurring conditions associated with Autism Spectrum
Disorder (ASD)? A: Individuals with ASD may experience difficulties with
```

verbal and nonverbal commun...

retrieved learned_qa [learned_qa]: Q: What are some common characteristics and challenges experienced by individuals with Autism Spectrum Disorder (ASD)? A: Individuals with ASD may experience difficulties with verbal and nonverbal com...

retrieved documents [..\data\pdfs\Autism spectrum disorder definition, epidemiology, causes, and clinical evaluation.pdf]: 11. de Giambattista C, Ventura P, Trerotoli P, et al. Subtyping the autism spectrum disorder: comparison of children with high functioning autism and asperger syndrome. J Autism Dev Disord 2019;49:138...

retrieved documents [..\data\pdfs\Autism spectrum disorder definition, epidemiology, causes, and clinical evaluation.pdf]: 53. Saxena A, Chahrour M. Autism spectrum disorder. In: Ginsburg GS, Willard HF, David SP. editors. Genomic and precision medicine: primary care. Cambridge: Academic Press, 2017:301-16.

- "ASD symptoms and causes"

retrieved learned_qa [learned_qa]: Q: What are some common co-occurring conditions with ASD? A: ASD can co-occur with other conditions, such as anxiety, ADHD, epilepsy, sleep disorders, or gastrointestinal issues. GI disorders or GI sy...

retrieved learned_qa [learned_qa]: Q: What are some common co-occurring conditions experienced by individuals with ASD? A: Many ASD patients experience co-occurring conditions such as anxiety, ADHD, epilepsy, sleep disorders, or gastro...

retrieved documents [..\data\word\ASD_Patient_Management_Handbook.docx]: Patients with ASD may experience difficulties with verbal and nonverbal communication, changes in routine, social interactions, emotional regulation, and sensory processing. Some individuals may avoid...

retrieved documents [..\data\word\ASD_Patient_Management_Handbook.docx]: Many ASD patients experience co-occurring conditions such as anxiety, ADHD, epilepsy, sleep disorders, or gastrointestinal issues. Healthcare providers should use patient-centered approaches and consi...

--- Prior failure #2 ---

Original query : what can u tell me about ASD?

User feedback : there was no precise information and it was very vague

Bad answer was : Autism Spectrum Disorder (ASD) is a complex neurodevelopmental condition characterized by challenges in social communication, restricted interests, repetitive behaviors, and sensory sensitivities [Source: learned_qa]. Individuals with ASD may experience difficulties with verbal and nonverbal communi...

Two independent prior failures for the same normalized question surface together, each with its own feedback text and its own per-reformulation retrieved-chunk previews — this is Chapter 31's thumbdown

mechanism made concrete: not a single "last bad answer" flag, but an accumulating, per-question failure history the model is shown in full before it tries again.

D.4 learned_qa — the self-learning ChromaDB collection

``get_or_create_learned_qa_collection()`` guarantees cosine distance regardless of how an older persisted collection was created, by snapshotting, recreating at cosine, restoring, and count-verifying before deleting the backup — with rollback on any failure. This is not a theoretical safeguard: Architecture.md records a completed production migration of 374 entries from L2 to cosine distance with every ID preserved.

```
combined = f"Q: {q}\nA: {a}"
uid = hashlib.sha256(combined.encode()).hexdigest()[:16]
metadatas.append({
    "source": "learned_qa",
    "original_query": interaction["query"][:200],
    "question": q[:300],
    "answer": a[:500],
    "interaction_ts": interaction.get("ts", ""),
})
```

Each distilled Q/A pair is deduplicated by a SHA-256-derived ID before it is written, so re-distilling the same text twice is a no-op rather than a duplicate entry. ``SelfLearner.run_distillation()`` labels learned-QA-track evidence ``[LEARNED QA - HIGH PRIORITY]`` inside its own distillation prompt, so a rephrasing distilled from an already-distilled answer preserves the same evidence hierarchy rather than flattening it. At retrieval time this collection is its own track — ``RETRIEVAL_TOP_L = 4`` results, ``LEARNED_QA_MIN_SIMILARITY = 0.57`` — merged alongside the ``documents`` track rather than replacing it.

D.5 rag_graph.png

A real, regenerated-on-change PNG rendering of the compiled LangGraph topology, written wherever ``app_workflow/main.py`` starts:

```
try:
    graph_image_path = Path(__file__).with_name("rag_graph.png")
    graph_image_path.write_bytes(rag_app.get_graph().draw_mermaid_png())
except ImportError:
    logger.info("Graph structure:\n%s", rag_app.get_graph().draw_ascii())
```

The rendered image is the two-track architecture the rest of this book documents made visible in one picture: retrieval fans out into parallel document and learned-QA tracks, each running its own dedup/merge → NAC → DC → LBC compression chain, converging at a ``combine_tracks`` barrier before drafting, quality-checking, and answering — with dashed retry edges looping back to query-variant generation. The status log records it being regenerated three separate times as the graph's topology visibly grew during development, which is as good a one-image summary of “this pipeline evolved” as the project has.

D.6 The debug-log pairs

Both pipelines write structured, per-request debug logs in the same format — ` | <LEVEL> | <logger> | [<request\_id>] | <message>` — with the bracketed request ID populated by a `ContextVar`-based filter so every line from one request, across every module it touches, can be isolated by grep even under concurrent traffic.

```
2026-07-10 11:58:41 | INFO          | api                  | [52820541-a502-4b63-b81e-7b58534bcf09] | Request received: request_id=52820541-a502-4b63-b81e-7b58534bcf09
2026-07-10 11:58:41 | DEBUG         | api                  | [52820541-a502-4b63-b81e-7b58534bcf09] | [BLOCKED VARIANTS] 2 known-failing variant(s) loaded from DB.
2026-07-10 11:58:41 | INFO          | agent_query          | [52820541-a502-4b63-b81e-7b58534bcf09] | [PHASE: RETRIEVE] Iteration 1/5
```

`langchain\_api\_\*.debug.log` carries one monolithic `agent\_query` logger for `app/`'s hand-rolled state machine; `langgraph\_api\_\*.debug.log` carries a per-node logger namespace (`nodes.retrieve`, `nodes.dc`, `nodes.combine\_tracks`, ...) reflecting `app\_workflow/`'s decomposition into one module per graph node. Both interleave raw MongoDB wire-level and outbound LLM request-body tracing at DEBUG level, which is what makes a single log file — not a diagram — the most complete record of one real run: it is the source Appendix B's annotated trace is built from.

D.7 old_log.txt and new_log.txt — a before/after regression pair

Unlike every other artifact in this appendix, these two are not code output — they are console captures the author took by hand on either side of a specific refactor (2026-06-15, the project's `print()`-to-`logger` migration) and attached directly to a conversation as evidence something had broken. `old\_log.txt` has no timestamps at all, pure `print()` output. The complete embedding-and-retrieval block it captures for the query "ASD", in full:

```
STEP: QUERY -> EMBEDDING
-----
Query       : "ASD"
Model       : all-MiniLM-L6-v2
Shape       : (384,)
First 8 vals : [-0.124032, -0.006127, -0.007644, 0.051178, -0.008589, -0.021051, 0.06904, -0.01971]
L2 norm      : 1.000000 (1.0 = fully normalised)
-----

Querying collections separately - documents top_k=5, learned_qa top_l=5,
score_threshold=0.5

Ranked 1 result(s) from documents after within-collection dedup/threshold
filter:
  documents[0] - score: 0.5424 [C:\Users\LOQ\Desktop\Projects\RAG-
work\data\json\asd_information.json]
                preview: Autism Spectrum Disorder (ASD) is a neurodevelopmental
```

condition characterized by differences in communication, social i...

Ranked 3 result(s) from learned_qa after within-collection dedup/threshold filter:

learned_qa[0] - score: 0.5144 [learned_qa]
preview: Q: What are the key characteristics of ASD in the context of engineering and technology? A: ASD is characterized by chal...
learned_qa[1] - score: 0.5131 [learned_qa]
preview: Q: What are the primary characteristics of ASD? A: ASD is a neurodevelopmental condition characterized by differences in...

`new\_log.txt` is the same pipeline, same iteration, now timestamped and level-tagged — but the equivalent moment (right after `[PHASE: RETRIEVE] Iteration 1/5` fires) contains no embedding block, no vector, no retrieval preview at all, only raw HTTP request/retry noise, in full:

```
2026-06-15 15:04:51 | INFO      | [PHASE: RETRIEVE] Iteration 1/5
2026-06-15 15:04:52 | INFO      | HTTP Request: POST
https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
2026-06-15 15:04:54 | INFO      | HTTP Request: POST
https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
2026-06-15 15:04:54 | INFO      | HTTP Request: POST
https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 429 Too Many
Requests"
2026-06-15 15:04:54 | INFO      | Retrying request to
/openai/v1/chat/completions in 6.000000 seconds
2026-06-15 15:05:02 | INFO      | HTTP Request: POST
https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
2026-06-15 15:05:02 | WARNING   | [VALIDATE-RETRIEVAL] failed to fix
malformed LLM output.
2026-06-15 15:05:02 | INFO      | HTTP Request: POST
https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 429 Too Many
Requests"
2026-06-15 15:05:02 | INFO      | Retrying request to
/openai/v1/chat/completions in 17.000000 seconds
2026-06-15 15:05:20 | INFO      | HTTP Request: POST
https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
2026-06-15 15:05:21 | INFO      | HTTP Request: POST
https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
2026-06-15 15:05:21 | WARNING   | [VALIDATE-RETRIEVAL] failed to fix
malformed LLM output.
2026-06-15 15:05:22 | INFO      | HTTP Request: POST
https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 429 Too Many
Requests"
2026-06-15 15:05:22 | INFO      | Retrying request to
/openai/v1/chat/completions in 23.000000 seconds
2026-06-15 15:05:47 | INFO      | HTTP Request: POST
```

```
https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
2026-06-15 15:05:47 | INFO | HTTP Request: POST
https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 429 Too Many
Requests"
2026-06-15 15:05:47 | INFO | Retrying request to
/openai/v1/chat/completions in 3.000000 seconds
2026-06-15 15:05:51 | INFO | HTTP Request: POST
https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
2026-06-15 15:05:51 | INFO | [CHUNK MERGE] success -- 332 chars, 2
source(s)
```

This pair is what surfaced BUG-038: the migration's regex matched inline ``print(...)`` calls but missed multi-line ``with open() as f: f.write(...)`` blocks, silently deleting entire categories of debug output (embedding vectors, retrieval previews, LLM input/output payloads, JSON-repair attempts) rather than converting them. The ``[VALIDATE-RETRIEVAL] failed to fix malformed LLM output`` warnings visible in ``new_log.txt`` above are themselves a second, unrelated logging bug (BUG-037, an inverted message on the success branch — see Appendix E's baseline stress-test run for the same message recurring) rather than evidence anything actually failed here.

Common pitfall — A refactor that passed every test and still lost data

``old_log.txt`` vs. ``new_log.txt`` is worth keeping as a technique, not just a historical anecdote: a mechanical, regex-driven refactor can silently drop content a test suite never checked for, because nothing failed — the program still ran, it just stopped recording what it used to record. Capturing a full run's raw console output before and after a refactor, then diffing it by eye, catches exactly this class of regression even with no formal test covering the missing content.